CONCURRENCY PROGRAMMING

By : Nguyen Ho (Eddy)

WHAT IS CONCURRENCY?

Concurrency is the ability of a system to execute multiple tasks during overlapping time periods by interleaving their.

- *Tasks do not need to run at the exact same time*
- *The CPU switches rapidly between tasks*
- *Each task makes progress over time*

WHY CONCURRENCY?

01

Improves performance

02

Enhances responsiveness

03

Handles multiple tasks efficiently

SEQUENTIAL EXECUTION

```java
class Bomb {
    private String name;

    public Bomb(String name) {
        this.name = name;
    }

    public void run() {
        for (int i = 1; i <= 10; i++) {
            System.out.println(name + ": " + i);
        }
        System.out.println(name + " exploded!");
    }
}

public class Main {
    public static void main(String[] args) {
        Bomb a = new Bomb("Bomb A");
        Bomb b = new Bomb("Bomb B");

        a.run();  // runs first
        b.run();  // runs after A finishes
    }
}
```

```
Bomb A: 1
Bomb A: 2
Bomb A: 3
...
Bomb A: 10
Bomb A exploded!

Bomb B: 1
Bomb B: 2
Bomb B: 3
...
Bomb B: 10
Bomb B exploded!
```

CONCURRENT EXCUTION

```
Bomb A: 1
Bomb B: 1
Bomb A: 2
Bomb B: 2
Bomb A: 3
Bomb A: 4
Bomb B: 3
Bomb B: 4
Bomb A: 5
Bomb B: 5
...
Bomb A exploded!
Bomb B exploded!
```

```java
class Bomb extends Thread {
    public Bomb(String name) {
        super(name);
    }

    public void run() {
        for (int i = 1; i <= 10; i++) {
            System.out.println(getName() + ": " + i);
        }
        System.out.println(getName() + " exploded!");
    }
}

public class Main {
    public static void main(String[] args) {
        Bomb a = new Bomb("Bomb A");
        Bomb b = new Bomb("Bomb B");

        a.start();
        b.start();
    }
}
```

Feature	Sequential Execution	Concurrent Execution
Execution	One task at a time	Multiple tasks overlap
Speed	Slower	Faster (in many cases)
Complexity	Simple	More complex
Output	Predictable	May be interleaved
Resource Usage	Low	Better utilization
Time (Big-O)	O(n + m)	O(max(n, m)) (ideal)

```java
class Bomb extends Thread {
    static int count = 0;

    public Bomb(String name) {
        super(name);
    }

    public void run() {
        for (int i = 0; i < 5; i++) {
            count++;
            System.out.println(getName() + ": " + count);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Bomb a = new Bomb("Bomb A");
        Bomb b = new Bomb("Bomb B");

        a.start();
        b.start();
    }
}
```

RACE CONDITION

```
Bomb A: 1
Bomb B: 1    ❌ (duplicate)
Bomb A: 2
Bomb B: 3
Bomb A: 3    ❌ (wrong order)
Bomb B: 4
Bomb A: 5
Bomb B: 6
...
```

RACE CONDITION

A race condition occurs when multiple threads access and modify shared data at the same time, leading to unpredictable results.

Using Our Bomb Example:

- Bomb A and Bomb B share the same counter
- Both try to update it at the same time
- The result becomes incorrect

SYNCHRONIZATION

Synchronization is used to control access to shared data, allowing only one thread to execute a critical section at a time.

Back to Bomb Example

- Bomb A and Bomb B share the same counter
- We lock the counter when updating
- Only one thread can update at a time

```java
class Bomb extends Thread {
    static int count = 0;

    public Bomb(String name) {
        super(name);
    }

    public void run() {
        for (int i = 0; i < 5; i++) {
            increase();
        }
    }

    public synchronized void increase() {
        count++;
        System.out.println(getName() + ": " + count);
    }
}
```

```
Bomb A: 1
Bomb B: 2
Bomb A: 3
Bomb B: 4
Bomb A: 5
Bomb B: 6
...
```

Applications of Concurrency

Web Server

Handle multiple users and requests at the same time.

Mobile Application

Run background tasks without blocking the main interface.

Games

Run physics, rendering, and input processing simultaneously.

AI & Data Processing

Train models and process large datasets faster with concurrent execution.

CONCLUSION

Key Takeaways

- Concurrency allows multiple tasks to run efficiently
- It improves performance and responsiveness
- But it introduces challenges like race conditions
- Synchronization is needed to ensure correctness

THANK YOU